Kasi Yedumati · Full Stack Training

Module 06 — Tuples and Sets

Phase: Core Python
Previous: Lists
Next: Dictionaries

Module Outcome

Seal an ordered record as a tuple, unpack it, convert list ↔ tuple, and use a set for unique values and fast membership.

By the end of this module you will be able to:

- Create a tuple as a sealed ordered record
- Avoid the `(42,)` one-item trap; index, slice, `count`, and `index`
- Unpack a tuple and swap two values
- Convert list ↔ tuple
- Build a set; write an empty set as `set()` — not `{}`
- Deduplicate a list; test membership with `in`
- Use light union (`|`) and intersection (`&`)

1. Why This Concept Matters

A list is the right bag when the collection should change. Some data should **not** change by accident — a printed receipt, a GPS drop, an employee snapshot. A **tuple** is an ordered, **immutable** sequence.

Imports are a different problem. Emails and product IDs often arrive twice. A **set** keeps each value **once** and answers "is this id already here?"

```
receipt  →  ("ORD-5001", 2499.0, "Delivered")   sealed
ids      →  {101, 102, 103}                      unique
```

2. Create a tuple

Understand

A tuple is written with parentheses `( )`. Order is kept. You cannot assign to an element.

Use it to freeze a receipt, GPS point, or catalog row so a typo cannot silently rewrite the record.

How It Works

```
employee = ("EMP-101", "Ananya Sharma", "Engineering")
print(employee)

receipt = ("ORD-5001", 2499.00, "Delivered")
print(f"Order {receipt[0]}: ₹{receipt[1]} — {receipt[2]}")
```

Expected Output:

```
('EMP-101', 'Ananya Sharma', 'Engineering')
Order ORD-5001: ₹2499.0 — Delivered
```

Parentheses are optional when packing: `order = "ORD-1", 999.0, "Paid"` is still a tuple.

⚠️ WATCH OUT

```
# Wrong — parentheses alone group an expression
single = (42)         # int, not a tuple

# Prefer — the trailing comma makes a 1-tuple
single = (42,)
```

Try It

Create `emp = ("EMP-204", "Priya Nair", "HR")`. Print each field by index on one line.

---

3. Read a tuple — you cannot edit

Understand

Indexing and slicing work like lists: `t[0]`, `t[-1]`, `t[1:3]`. Helpers: `.count(x)`, `.index(x)`, `len(t)`.

Assignment to an element raises `TypeError`. To "change" a field, build a **new** tuple.

How It Works

```
scores = (78, 92, 85, 92, 88)
print(scores[0])
print(scores[-1])
print(scores.count(92))
print(scores.index(85))

shipment = ("ORD-9", 1499.0, "Packed")
print(len(shipment))
# shipment[2] = "Shipped"   # TypeError — leave commented

one = (100,)
not_tuple = (100)
print(type(one), type(not_tuple))
```

Expected Output:

```
78
88
2
2
3
<class 'tuple'> <class 'int'>
```

⚠ WATCH OUT

```
# Wrong
employee[0] = "EMP-999"     # TypeError

# Prefer — new tuple
employee = ("EMP-999",) + employee[1:]
```

A tuple of immutables can later be a **dict key**. A list cannot. That matters in the next module — remember the rule, do not build dicts here.

Try It

Compare `one = (100,)` and `not_tuple = (100)`. Print both `type(...)` results.

---

4. Unpack and swap

Understand

Unpacking names the fields: `order_id, amount, status = order`. The number of names must match the length.

Swap is one line: `a, b = b, a`. No temp variable.

```
order_summary  →  ("ORD-5001", 2499.0, "Delivered")
                        ↓ unpack
order_id="ORD-5001"  amount=2499.0  status="Delivered"
```

How It Works

```
a, b = "Ravi", "Priya"
a, b = b, a
print(a, b)

date, category, amount = ("31-07-2026", "Food", 450.0)
print(f"{date} | {category} | ₹{amount}")
```

Expected Output:

```
Priya Ravi
31-07-2026 | Food | ₹450.0
```

⚠ WATCH OUT

```
# Wrong — count does not match
x, y = (1, 2, 3)      # ValueError

# Prefer
x, y, z = (1, 2, 3)
```

Try It

Swap `left = "Tea"` and `right = "Coffee"`, then print both. Unpack `("P010", "Chai Mug", 299)` into `sku`, `name`, `price` and print the name.

---

5. list ↔ tuple

Understand

`tuple(lst)` seals a list. `list(t)` opens a tuple for editing again.

Lists are editable carts. Tuples are sealed snapshots. Convert when the job changes.

A list of tuples is a common catalog: `(sku, name, price)` rows you unpack in a loop.

How It Works

```
ids = [101, 102, 103]
sealed = tuple(ids)
print(sealed)
print(list(sealed))

catalog = [
    ("P001", "Wireless Mouse", 799),
    ("P002", "Keyboard", 1499),
    ("P003", "USB Hub", 599),
]
for sku, name, price in catalog:
    print(f"{sku}: {name} — ₹{price}")
```

Expected Output:

```
(101, 102, 103)
[101, 102, 103]
P001: Wireless Mouse — ₹799
P002: Keyboard — ₹1499
P003: USB Hub — ₹599
```

⚠ WATCH OUT

Converting to a tuple does **not** protect a mutable item *inside* (a list nested in a tuple can still be edited). Prefer simple hashable fields (`str`, `int`) in core work.

Try It

Convert marks `[78, 92, 85]` to a tuple, then back to a list. Print both.

---

6. Create a set — uniqueness

Understand

A **set** is an unordered collection of **unique** hashable items. Duplicates are dropped.

Empty set is `set()` — **not** `{}` (`{}` is an empty dict, next module).

Signups, tags, and product IDs often arrive with duplicates. A set keeps each value once before import.

How It Works

```
skills = {"Python", "SQL", "Git", "Python"}
print(sorted(skills))

empty = set()
print(type(empty))

raw_ids = [101, 102, 101, 103, 102, 104]
unique_ids = set(raw_ids)
print("Unique count:", len(unique_ids))
print(sorted(unique_ids))
```

Expected Output:

```
['Git', 'Python', 'SQL']
<class 'set'>
Unique count: 4
[101, 102, 103, 104]
```

`sorted(...)` is for stable display. A set itself has **no** promised print order.

Grow a set with `.add(x)`. Adding a value that is already there is ignored.

```
tags = {"sale"}
tags.add("gift")
tags.add("sale")
print(sorted(tags))
```

Expected Output:

```
['gift', 'sale']
```

⚠️ WATCH OUT

```
# Wrong — empty dict, not empty set
empty = {}            # dict

# Wrong — list is unhashable
{[1, 2]}              # TypeError

# Prefer
empty = set()
{1, 2}                # ints are hashable
```

Print order of a set is not guaranteed — use `sorted(...)` in exams and logs.

Try It

From `[1, 2, 2, 3, 3, 3]`, build a set. Print `len` before and after, then `sorted(unique)`.

---

7. Membership with `in`

Understand

`"Python" in skills` asks whether the value is a member. On a set this is typically fast — ideal for “is this tag / ID allowed?”

How It Works

```
skills = {"Python", "SQL", "Git"}
print("Python" in skills)
print("Java" in skills)

allowed = {"electronics", "sale", "gift"}
tag = "sale"
if tag in allowed:
    print("Tag OK")
else:
    print("Unknown tag")
```

Expected Output:

```
True
False
Tag OK
```

⚠️ WATCH OUT

Sets are **unordered** — never depend on print order. Prefer `sorted(set(...))` when you need sorted unique output.

Try It

Given `allowed = {"chai", "samosa", "dosa"}`, print whether `"idli"` is allowed and whether `"chai"` is allowed.

---

8. Union and intersection

Understand

Op	Meaning	Symbol
Union	In either set	`a | b`
Intersection	In both	`a & b`

Two batches or two tag lists — “everyone” vs “common only.”

How It Works

```
batch_a = {"Priya", "Ravi", "Dev"}
batch_b = {"Ravi", "Meera"}
```

```
print(sorted(batch_a | batch_b))
print(sorted(batch_a & batch_b))
```

Expected Output:

```
['Dev', 'Meera', 'Priya', 'Ravi']
['Ravi']
```

When you start from lists: `set(list_a) & set(list_b)`.

⚠ WATCH OUT

`|` and `&` need **sets** (or compatible set types). `["A"] | ["B"]` is not valid — convert first.

Try It

Given `batch_2024 = ["A", "B", "C", "D"]` and `batch_2025 = ["C", "D", "E", "F"]`, print students in **both** batches (sorted).

Step-by-Step: Build It

One assemble: seal a catalog row as a tuple, unpack it, then dedupe an import id list with a set.

`dict.fromkeys` below is a **one-line preview** — dictionaries keep insertion order, so this rebuilds a unique list in first-seen order. Full dicts are the next module. You need it only because a set does not keep order.

```
row = ("P010", "Chai Mug", 299)
sku, name, price = row
print(f"{sku}: {name} — ₹{price}")

raw = "101, 102, 101, 103, 102, 104"
parts = []
for piece in raw.split(","):
    token = piece.strip()
    if token:
        parts.append(token)

unique_set = set(parts)
print("Original count:", len(parts))
print("Unique count:", len(unique_set))
print("Set (sorted for display):", sorted(unique_set))

# Preview only — first-seen order (dict keeps insertion order)
clean = list(dict.fromkeys(parts))
print("Clean list (order kept):", clean)
```

Expected Output:

```
P010: Chai Mug — ₹299
Original count: 6
Unique count: 4
Set (sorted for display): ['101', '102', '103', '104']
Clean list (order kept): ['101', '102', '103', '104']
```

Empty tokens after split must be skipped — a trailing comma would otherwise become `""`.

Practice Exercises

Attempt each problem before reading the model answer.

1. Create `emp = ("EMP-204", "Priya Nair", "HR")`. Print each field by index on one line.

Model answer:

```
emp = ("EMP-204", "Priya Nair", "HR")
print(emp[0], emp[1], emp[2])
```

Expected Output:

```
EMP-204 Priya Nair HR
```

2. Compare `one = (100,)` and `not_tuple = (100)`. Print both `type(...)` results.

Model answer:

```
one = (100,)
not_tuple = (100)
print(type(one), type(not_tuple))
```

Expected Output:

```
<class 'tuple'> <class 'int'>
```

3. Convert marks `[78, 92, 85]` to a tuple, then back to a list. Print both.

Model answer:

```
marks = [78, 92, 85]
t = tuple(marks)
print(t)
print(list(t))
```

Expected Output:

```
(78, 92, 85)
[78, 92, 85]
```

4. From `[1, 2, 2, 3, 3, 3]`, build a set. Print `len` before and after, then `sorted(unique)`.

Model answer:

```
nums = [1, 2, 2, 3, 3, 3]
print(len(nums))
unique = set(nums)
print(len(unique))
print(sorted(unique))
```

Expected Output:

```
6
3
[1, 2, 3]
```

5. Given `batch_2024 = ["A", "B", "C", "D"]` and `batch_2025 = ["C", "D", "E", "F"]`, print students in **both** batches (sorted).

Model answer:

```
batch_2024 = ["A", "B", "C", "D"]
batch_2025 = ["C", "D", "E", "F"]
both = set(batch_2024) & set(batch_2025)
print(sorted(both))
```

Expected Output:

```
['C', 'D']
```

6. Seal `("ORD-9", 1499.0, "Packed")`, unpack into three names, and print `ORD-9 — Packed`. Then add `"Packed"` and `"Shipped"` to a set of statuses and print whether `"Delivered"` is in the set.

Model answer:

```
order_id, amount, status = ("ORD-9", 1499.0, "Packed")
print(f"{order_id} — {status}")

statuses = {"Packed", "Shipped"}
print("Delivered" in statuses)
```

Expected Output:

```
ORD-9 — Packed
False
```

Lab

Combine a sealed tuple, unpacking, and set uniqueness / membership. No `def`, no files. Do not turn this into the comma-separated CLI from the kit.

Problem

A catalog import has a sealed product row plus a messy id list. You must print the product line, unique ids, and whether a sku tag is allowed.

What to Build

A short script that unpacks one tuple, dedupes a list into a set, and checks membership.

Requirements

- Seed `row = ("P010", "Chai Mug", 299)` and unpack into `sku`, `name`, `price`
- Print `sku: name — ₹price`
- From `[101, 102, 101, 103, 102, 104]`, print original count, unique count, and `sorted(unique)`
- `allowed = {"mug", "bottle", "notebook"}` — print whether `"mug"` is allowed and whether `"pen"` is allowed
- Two batches `{"Priya", "Ravi"}` and `{"Ravi", "Dev"}` — print the intersection sorted

Expected Behavior

Unique count is 4. `"mug"` is allowed; `"pen"` is not. Intersection is `['Ravi']`.

Sample Run

```
P010: Chai Mug — ₹299
Original count: 6
Unique count: 4
[101, 102, 103, 104]
True
False
['Ravi']
```

(Your labels may differ; the **shape** should match.)

Constraints

Top-level statements only — no `def`, no type hints, no files. Empty set, if you need one, is `set()` not `{}`.

Bonus

Print a first-seen unique list with `list(dict.fromkeys(...))` and label it as a preview — not a dict lesson.

Mini-Project (Assignment): Email / Product ID Dedup Tool

Why this matters: Imports arrive with duplicate ids. Unique count plus a clean list is the same job as cleaning a mailing list or SKU file before a database load.

Lab kit: 15_Lab_Assets/31_email_id_dedup/

File	What you do
`starter/dedup.py`	Split, strip, skip empties; print counts + clean list
`broken/dedup.py`	Loses order or crashes on empty tokens — fix both
`solution/`	Do **not** copy this

Objective

Read a comma-separated line of ids, skip empty tokens, and print original count, unique count, and a clean list in **first-seen** order.

Story / Scenario

Ops paste `101, 102, 101, 103`. Duplicate ids must be stripped before import — without requiring functions yet.

Requirements

Piece	Detail
Prompt	`Enter IDs comma-separated:`
Tokens	Split on `,`, `.strip()` each piece, skip empty
Counts	Print **original count** and **unique count**
Clean list	First-seen order (`list(dict.fromkeys(...))` is the kit preview — not a dict lesson)
Style	Top-level script — **no `def` required**, no type hints

Expected Behavior

- Empty tokens after split are ignored
- Unique count drops duplicates
- Clean list keeps the first time each id appeared

Sample Run

```
=== Dedup Tool ===
Enter IDs comma-separated: 101, 102, 101, 103, 102, 104
Original count: 6
Unique count: 4
Clean list: ['101', '102', '103', '104']
```

Bonus Challenges

- Second mode: unique via `set` (order may differ — say so in the print)
- Case-insensitive email dedupe while keeping first-seen casing
- **Later:** Data Structures Overview — when to pick list vs tuple vs set vs dict

Done when

- [ ] `101,102,101,103` → unique count 3, first-seen order preserved
- [ ] Empty tokens after split are ignored
- [ ] Original vs unique counts both print
- [ ] Optional set mode may change order (documented)
- [ ] No `def` / type hints on the core path

Interview Q&A

Q1. List vs tuple?

List is mutable; tuple is immutable. A tuple can be a dict key if all elements are hashable; a list cannot.

Q2. Why does `(42)` fail as a 1-tuple?

Parentheses alone group an expression. Need the comma: `(42,)`.

Q3. How do you swap two variables in Python?

`a, b = b, a` (tuple unpacking).

Q4. Empty set — `{}` or `set()`?

`set()`. `{}` creates an empty **dict**.

Q5. Are sets ordered in Python 3.7+?

No. Sets are unordered by language rules. Dicts keep insertion order; that does **not** apply to sets.

Q6. Convert list ↔ tuple?

`tuple(lst)`, `list(t)`.

Q7. Why can a tuple be a dict key but a list cannot?

Keys must be hashable. Lists are mutable; their contents (and hash) could change.

Q8. What is `frozenset`?

An **immutable** set — usable as a dict key or as an element inside another set. Know the name for interviews; not required in tonight's lab.

Cheat Sheet

```
(1, 2, 3)           → tuple (sealed, ordered)
(42,)               → 1-tuple (comma required)
t[i] / t[a:b]       → read / slice (no t[i] = ...)
t.count / t.index / len(t)
a, b = t            → unpack (length must match)
a, b = b, a         → swap
tuple(lst) / list(t)→ convert
{1, 2, 2}           → set {1, 2}  (unique, unordered)
set()               → empty set  ({} is an empty dict)
s.add(x)            → add (ignored if already present)
x in s              → membership
a | b               → union
a & b               → intersection
sorted(s)           → stable display
```

Sets drop duplicates. Use `sorted` when output order must be stable. First-seen unique lists: `list(dict.fromkeys(items))` — preview only.